Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I will use Drafter notation, For loops, Dataclasses, Function definitions, lists, and calling portions of databases the most. I would like to improve my skills on using data from other files in this project

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest part of the project will be the function definitions for the math portion of finding the molar masses. This is because the math functions of python are very easy for me to understand. The hardest part for me will be calling data from other files and turning it into a mutable float. This is because I am unsure in my abilities of both converting data types and calling from a third party source

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think laying out the foundation (i.e dataclasses, formatting, format function definition, etc) will take the longest because it is what the entire project is going to be built upon, so it will be the hardest to build with no bugs. It should also be the first implemented for the same reason.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Other sites on my computer are the largest distraction and I can minimize this through utilizing thonny in full screen. I do not have any questions on the requirements

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**Gives the user an easily accessible calculator with resources to help the user generate a function such as common charges of elements and molar masses.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	Slass State: elements: list[Element] calculations: list[Calculation] current_formula: string current_result: string selected_calculation: string
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Class: Element Attributes: element_name: String atom_num: integer mol_mass: float symbol: str Class: Calculation: Attributes: formula: string calculation_type: string

	result: string
Routes (minimum 7) The names of your route functions	home_page, calculator_page, calculate_formula, periodic_table_page, element_details_page, polyatomic_ions_page, history_page, clear_history.
Pages (minimum 5) A brief description of the pages in your site	**Title Page**- gives the page a welcoming front page and offers help of how to use it, as well as access to the rest of the site **Help Page**- gives instructions of how to use the site efficiently **Calculator Page-** gives access to the ion calculator **Calculation History Page-** shows past calculations with function **Polyatomic Page**- list of common polyatomics **Elemental list-** reference page of elements and their information
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	TextBox for chemical formula input. SelectBox for choosing the calculation type. TextBox for searching periodic table elements. SelectBox for filtering polyatomic ions by charge SelectBox for changing pages.
If Statements (minimum 3)	Check whether the chemical formula is valid.

Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Check which calculation type was selected. Check whether an element matches the search. Check whether calculation history is empty. Check whether a selected element exists.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	A for loop iterates through state.elements to display each element in the periodic table. Another possible loop iterates through state.calculations to display previous calculations.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Requirement	Implementation
Purpose/Functionality	

A 1-2 sentence description of the purpose or functionality of your site

---

State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.

Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)

Routes (minimum 7)

The names of your route functions

Pages (minimum 5)

A brief description of the pages in your site

User Input (minimum 3)

Description of user input components (TextBox, TextArea,

CheckBox, or SelectBox) and their purposes

For more information on these input types, see .

If Statements (minimum 3)

Brief description of what will be evaluated in your if

statements and how they will meaningfully change the

logic of the site

Loop (minimum 1)

This loop must iterate over the list/dict of dataclass instances in your State attribute.

Description of this loop and why it is meaningful to the site

help me name these and find where I will use them for this project:

Website that takes in chemical formulas and gives the option to find the atomic number of the whole molecule, atomic mass of a part of it, or the charge of the molecule. It will also have a periodic table page, where the user can find basic information about elements from an interactive list of elements and common polyatomics.

...

Your idea would work really well as a chemistry reference and calculator website! I would organize it around two main dataclasses: Element and Calculation.

The `Element` dataclass stores information about periodic table elements, while the `Calculation` dataclass stores the chemical formula calculations users perform.

Below is a suggested plan that meets all the requirements in your worksheet while keeping the project realistic for a CS1 student using Drafter.

1. Project name ideas

Some names that could work:

- ChemCalc – Simple and straightforward.
- MoleculeMate – Friendly chemistry assistant.
- Chemistry Central – A general chemistry reference.
- Formula Finder – Focused on chemical formulas.
- AtomIQ – Short and catchy.

- Chemistry Companion – A tool for students learning chemistry.

My recommendation is ChemCalc because it clearly communicates what the website does.

2. Dataclasses

I recommend these two dataclasses:

First dataclass: Element

This represents an individual element on the periodic table.

```
@dataclass

class Element:

    name: str

    symbol: str
```

```
    atomic_number: int

    atomic_mass: float

    common_charge: int
```

Attribute	Type	Purpose
name	str	Full element name, such as Oxygen.
symbol	str	Chemical symbol, such as O.
atomic_number	int	Number of protons in the element.
atomic_mass	float	Atomic mass of the element.
common_charge	int	A commonly occurring ionic charge, if applicable.

These attributes would mostly be initialized when the website starts. They don't need to change because element information is relatively constant.

Second dataclass: Calculation

This represents a calculation performed by the user.

```
@dataclass

class Calculation:

    formula: str

    calculation_type: str

    result: str
```

Attribute	Type	Purpose

formula	str	The chemical formula entered by the user.
calculation_type	str	The type of calculation (atomic number, atomic mass, or charge).
result	str	The calculated answer displayed to the user.

These attributes will be modified whenever the user performs a new calculation. Each completed calculation can be added to a list in the State dataclass.

3. State dataclass

Your State needs at least four attributes. Here is a suggested setup:

```
@dataclass

class State:

    elements: list[Element]
```

```
    calculations: list[Calculation]

    current_formula: str

    current_result: str

    selected_calculation: str
```

Attribute	Type	How it is used and modified
elements	list[Element]	Stores all periodic table elements. Used to display element information and search for elements.
calculations	list[Calculation]	Stores the user's previous calculations. A new Calculation is added whenever the user completes a calculation.
current_formula	str	Stores the chemical formula currently entered by the user. Changes whenever the user enters a new formula.
current_result	str	Stores the result of the most recent calculation. Changes after each calculation.

selected_calculation	str	Stores which calculation the user wants to perform (atomic number, atomic mass, or charge). Changes when the user selects a calculation type.

Why this works: You have five State attributes, and `elements` and `calculations` both contain lists of instances of your dataclasses.

4. Routes (minimum 7)

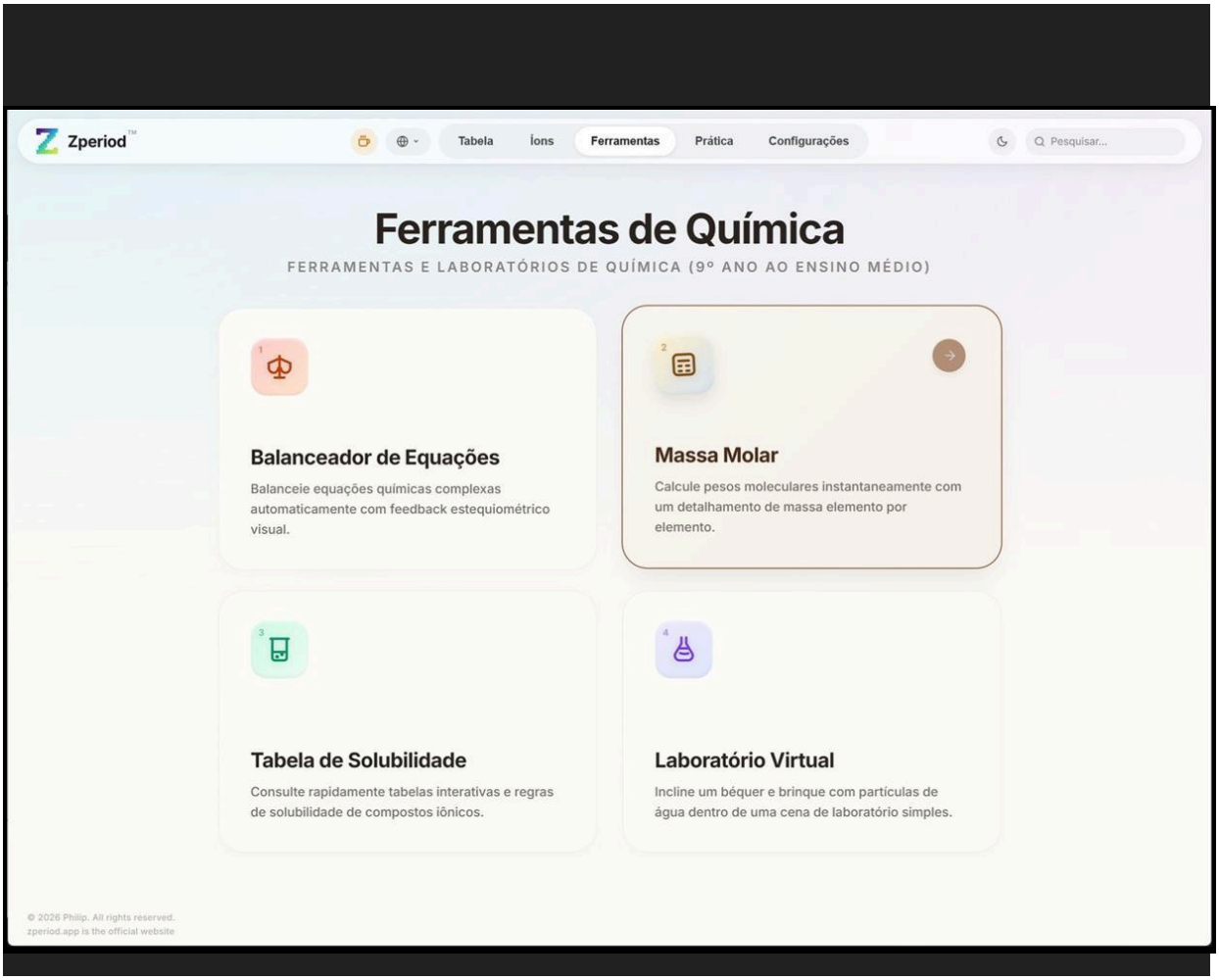
Here are eight possible route functions:

Route function	Purpose
`home_page`	Displays the homepage and introduces the website.
`calculator_page`	Displays the chemical formula calculator.
`calculate_formula`	Processes the chemical formula and calculates the requested result.

`periodic_table_page`	Displays the periodic table element list.
`element_details_page`	Displays detailed information about a selected element.
`polyatomic_ions_page`	Displays a list of common polyatomic ions and their formulas and charges.
`history_page`	Displays the user's previous calculations.
`clear_history`	Clears the user's calculation history.

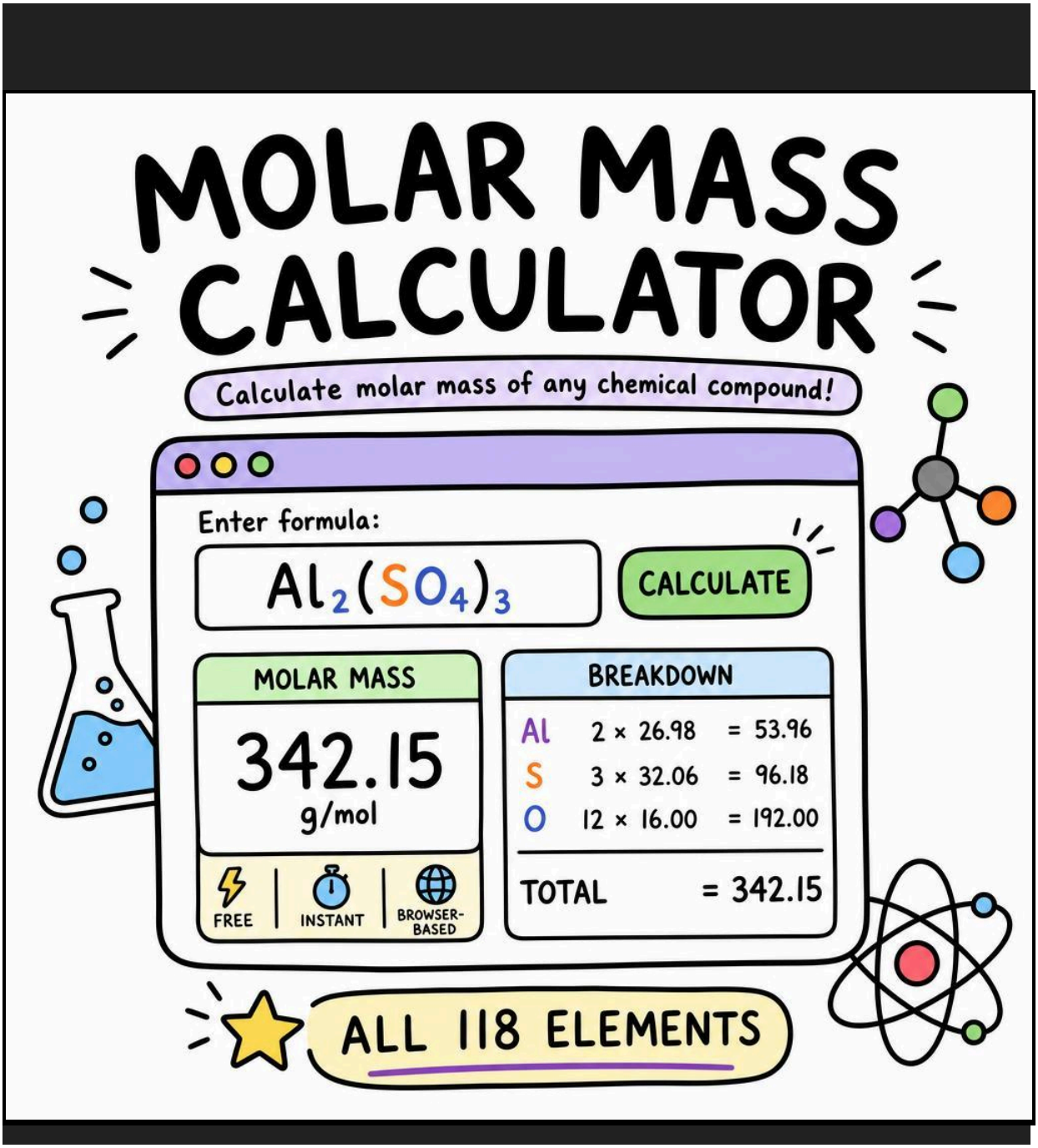
5. Pages (minimum 5)

I would organize the website into these five main pages:



1. Home Page

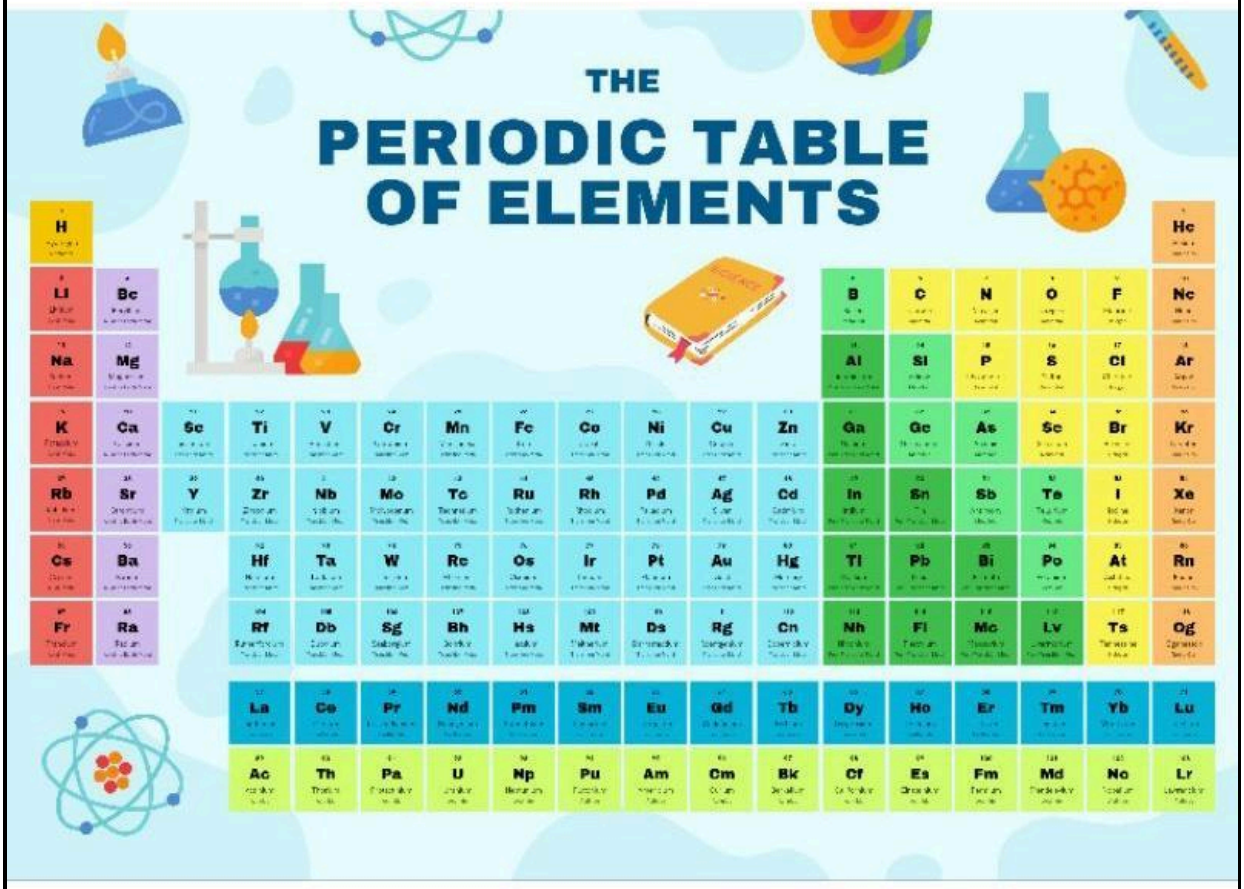
Introduces ChemCalc and provides navigation to the calculator, periodic table, and polyatomic ion reference.



2. Calculator Page

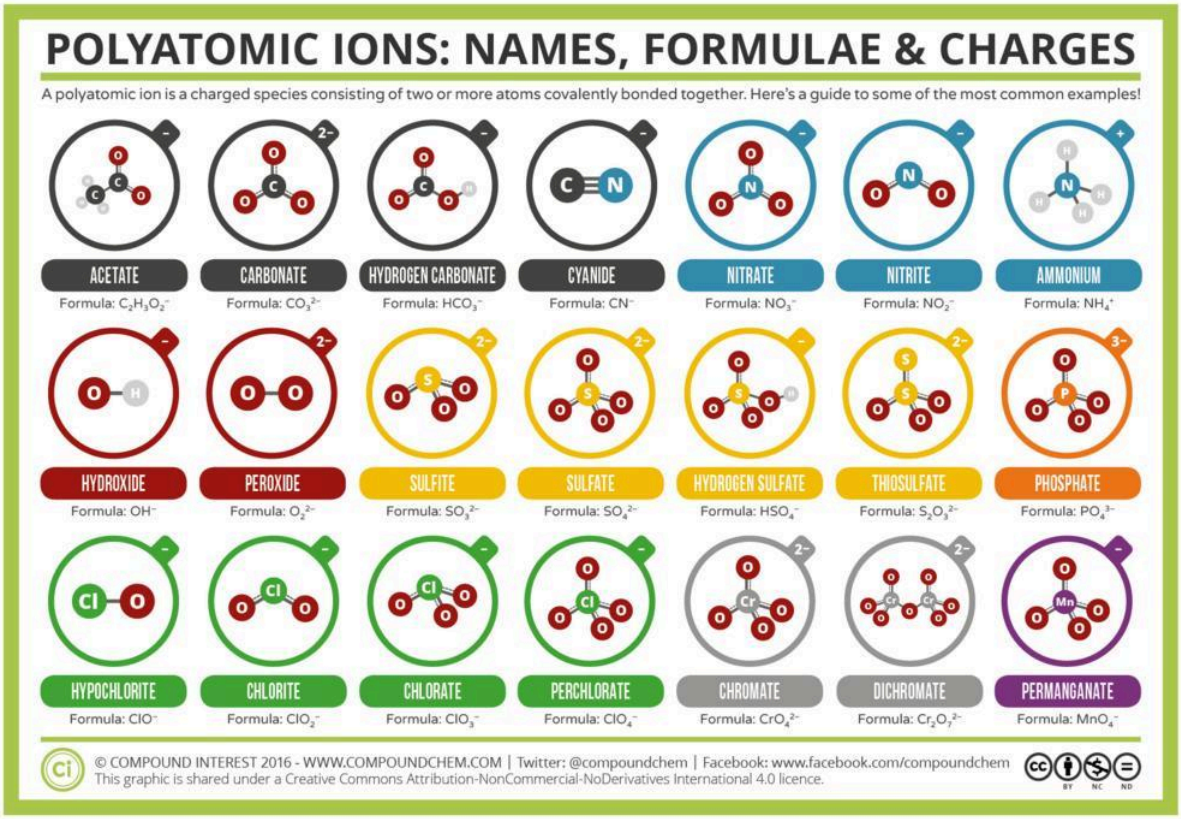
Allows users to enter a chemical formula and choose between calculating atomic

mass, total atomic number, or net charge.



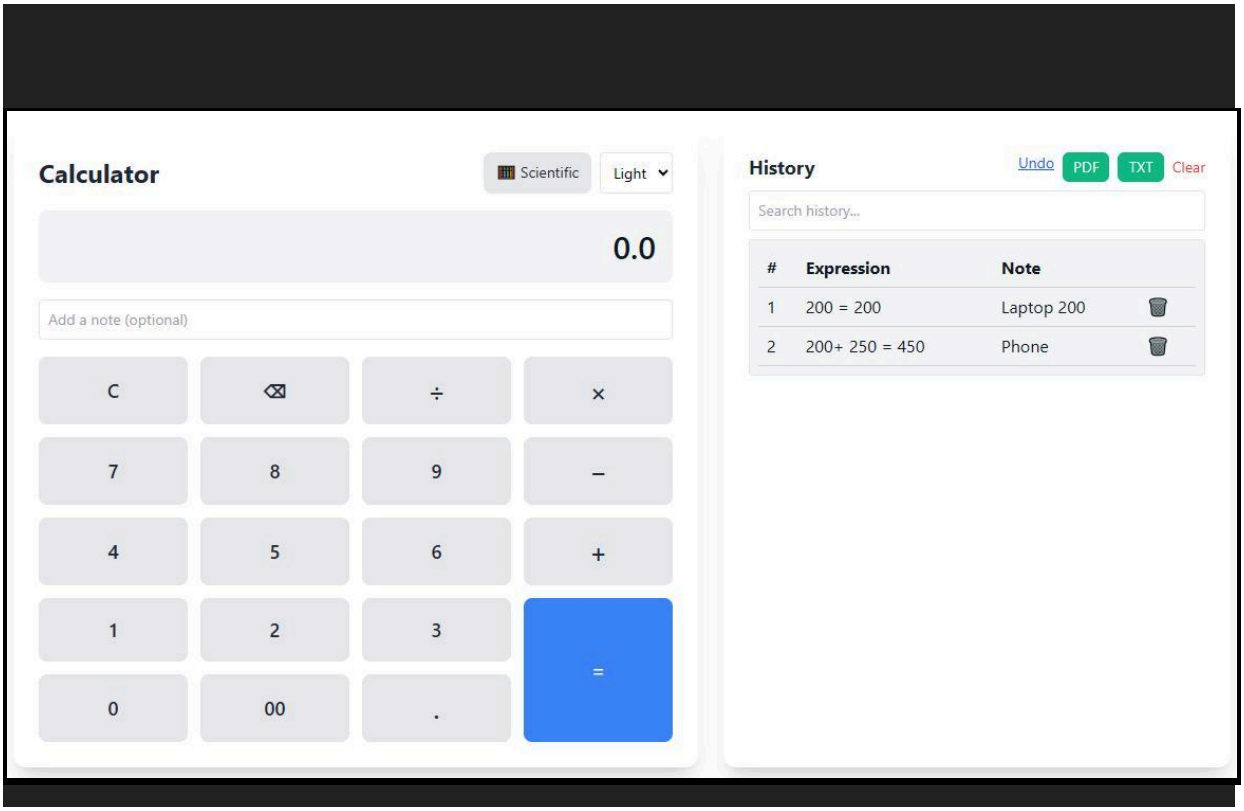
3. Periodic Table Page

Displays an interactive list of elements. Selecting an element displays its name, symbol, atomic number, atomic mass, and common charge.



4. Polyatomic Ions Page

Displays common polyatomic ions, their formulas, names, and charges.



5. Calculation History Page

Shows previous formulas, the type of calculation performed, and the results.

6. User Input (minimum 3)

You can use Drafter's input components for these.

Compone	Where you use it	Purpose

nt

TextBox	Calculator Page	Allows users to enter a chemical formula, such as H2O or Ca(OH)2.
SelectBox	Calculator Page	Allows users to choose atomic number, atomic mass, or charge.
TextBox	Periodic Table Page	Allows users to search for an element by its name or symbol.
SelectBox	Polyatomic Ions Page	Allows users to filter ions by charge.

You technically have four input components here, exceeding the minimum of three.

7. If Statements (minimum 3)

Here are five meaningful examples:

If statement	What it evaluates	How it changes the site
Formula validation	If the user entered a valid chemical formula.	Displays an error message if the formula is invalid instead of calculating.
Calculation selection	If the user selected atomic mass, atomic number, or charge.	Runs the appropriate calculation.
Element search	If an element's name or symbol matches the search input.	Displays matching elements.
Calculation history	If the calculation history is empty.	Displays a message saying there are no previous calculations.
Element selection	If the selected element exists in the element list.	Displays its information or an error if it cannot be found.

8. Loop (minimum 1)

You can use a `for` loop to display the elements stored in your State dataclass.

Example:

```
for element in state.elements:

    # Display element name

    # Display element symbol

    # Display atomic number
```

Where: Periodic Table Page.

Purpose: Iterates through the list of Element instances and displays each element's information. This allows you to display the entire periodic table without manually creating a separate section for every element.

You could also use another loop on the Calculation History Page:

```
for calculation in state.calculations:
```

```
    # Display formula


    # Display calculation type


    # Display result
```

This displays all the user's previous calculations.

9. Important considerations about the chemistry calculations

There is one technical detail I would strongly recommend addressing before you start coding.

Your three calculations have different levels of difficulty.

Calculation	Difficulty	How it works
Atomic mass	Easier	Multiply each element's atomic mass by its subscript and add the results.

Total atomic number	Easier	Multiply each element's atomic number by its subscript and add the results.
Molecular charge	More difficult	Requires knowing the charges of individual ions or atoms and accounting for their quantities.

One important distinction: Atomic number is technically defined for an individual element, not a molecule. You can calculate the *total number of protons* in a molecule by adding the atomic numbers of all its atoms. I would label that feature "Total Protons" to avoid confusing users.

Also, a chemical formula alone does not always tell you its charge. For example, you may need to recognize a known polyatomic ion or ask the user to provide the charge.

My recommendation for keeping the project manageable

Start by supporting:

- Basic formulas: H_2O, CO_2, $NaCl$, NH_3.
- Parentheses: $Ca(OH)_2$, if you have time.

- Atomic mass calculations.
- Total proton calculations.
- A reference list of common ionic charges and polyatomic ions.

Avoid trying to build a chemistry engine that understands every possible chemical formula. That would be much too ambitious for an introductory programming project.

One more Drafter consideration: Since Drafter runs Python in the browser without a backend or database, you can store the element and ion reference data directly in Python lists. You can also use browser local storage if you want calculation history to persist after the user refreshes the page.

10. Copy-ready version for your worksheet

Here is a condensed version you can adapt directly into your assignment.

Requirement	Implementation
Purpose/Functionality	ChemCalc is a chemistry calculator and reference website that allows users to enter chemical formulas and calculate their total number of protons, atomic mass, or molecular charge. It also includes an interactive periodic table, a polyatomic ion reference, and a calculation history.

State Dataclass Attributes	`elements: list[Element]` stores periodic table information. `calculations: list[Calculation]` stores previous calculations. `current_formula: str` stores the current formula. `current_result: str` stores the latest result. `selected_calculation: str` stores the selected calculation type.
Second Dataclass	`Calculation`: `formula: str` stores the chemical formula, `calculation_type: str` stores the calculation selected, and `result: str` stores the answer.
Routes	`home_page`, `calculator_page`, `calculate_formula`, `periodic_table_page`, `element_details_page`, `polyatomic_ions_page`, `history_page`, `clear_history`.
Pages	Home Page: Website introduction and navigation. Calculator Page: Formula input and calculations. Periodic Table Page: Interactive element reference. Polyatomic Ions Page: Common ion reference. Calculation History Page: Previous calculations.
User Input	TextBox for chemical formula input. SelectBox for choosing the calculation type. TextBox for searching periodic table elements. SelectBox for filtering polyatomic ions by charge.

If Statements	Check whether the chemical formula is valid. Check which calculation type was selected. Check whether an element matches the search. Check whether calculation history is empty. Check whether a selected element exists.
Loop	A for loop iterates through `state.elements` to display each element in the periodic table. Another possible loop iterates through `state.calculations` to display previous calculations.

My biggest suggestion: Make the calculator the centerpiece of the website, and treat the periodic table and polyatomic ion pages as reference tools that support it. That gives your application a clear purpose rather than making it feel like several unrelated chemistry pages.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.